
From sequences to schemas: How recurrent neural nets learn temporal abstractions

Anonymous Authors¹

Abstract

The ability to extract abstract relational structures from temporal sequences – independent of specific token identities – is a hallmark of intelligent behavior. Here, we investigate the neural mechanisms underlying this ability using Recurrent Neural Networks (RNNs) trained to classify sequences based on their latent algebraic patterns (e.g., $aab, aad \rightarrow AAB$; $aba, aca \rightarrow ABA$) without intermediate supervision. We demonstrate that RNNs spontaneously learn low-dimensional representations that mirror the hierarchical generative structure of the input data. Mechanistic analysis reveals that this abstraction is supported by the emergence of low-rank recurrent connectivity, driving the formation of a structured, tree-like geometry in the hidden state. Through singular vector ablation, we show that the leading singular component of the recurrent weights specifically integrates the relational transition information (same vs. different token) over time. Furthermore, while RNNs trained on next-token prediction do not spontaneously acquire these abstract representations, transferring the low-rank scaffold learned from classification significantly accelerates learning and improves generalization in prediction tasks. Our findings provide a mechanistic account of how task constraints shape low-rank recurrent dynamics to support temporal abstraction and representational transfer.

1. Introduction

The ability to infer abstract rules from experience is a cornerstone of intelligent behavior, enabling predictions, generalization and flexible behavior. Many real-world processes unfold as structured sequences, and brains have evolved

to detect and exploit sequential regularities across multiple levels of abstraction: from simple transitions and timing, to chunking, ordinal structure, and rule-based patterns, often organized into hierarchical schemas (Dehaene et al., 2015). Despite its importance, the neural mechanisms supporting abstraction and generalization remain elusive (Murphy et al., 2008; Santolin & Saffran, 2018). A particularly revealing form of abstraction is sensitivity to algebraic temporal patterns, such as AAB or ABA which capture relational structure independent of sensory identity (e.g., sounds, colors, shapes, etc.) (Marcus, 2003). Sensitivity to such patterns has been observed across species, including humans (Wang et al., 2015; Wu et al., 2025b), non-human primates (Wang et al., 2015), rodents (Murphy et al., 2008; Urcelay & Miller, 2010), and birds (Santolin et al., 2016; Spierings & Ten Cate, 2016; Mikolasch et al., 2013). How do neural circuits learn and represent these abstract temporal schemas (Dehaene et al., 2015), and what computational principles support their generalization across stimuli and contexts?

Recurrent Neural Networks (RNNs) are well suited to modeling time-evolving processes like working memory, context integration, and decision making (Machens et al., 2005), and are increasingly used to study how neural dynamics support behavior (Barak, 2017; Yang & Molano-Mazón, 2021; Driscoll et al., 2022). Here, we use RNNs to investigate the emergence of abstract temporal structures from sequential data, and to identify circuit-level mechanisms that enable generalization.

We systematically generate algebraic sequences using a binary branching tree, whose terminal nodes define abstract sequence classes (denoted by capital letters in this manuscript, e.g., AAB or ABA) of a certain length, and individual letters (tokens) sampled from an alphabet of a given size. We train RNNs to classify sequences according to their overall abstract pattern, receiving only the class label at the end of the sequence (e.g., AAB for aac and bba ; ABA for aca and ada), without any supervision on intermediate transitions.

We show that trained networks generalize to novel token instantiations and develop low-dimensional hidden representations whose geometry mirrors the branching structure of the generative tree. We then establish a mechanistic link between representation and circuit structure: recurrent connec-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

tivity becomes effectively low-rank, and the leading singular components causally support abstraction by integrating transition history over time, as revealed by rank-constrained training and singular-vector ablations. Finally, we compare task objectives and test reuse of learned structures: next-token prediction trained from scratch does not yield the same hierarchical, low-rank organization, but initializing prediction models with classification-trained recurrent weights accelerates learning and improves generalization. Crucially, this benefit is specific to the learned abstract scaffold: control experiments using autoencoder pretraining fail to improve generalization, indicating that the transfer gain arises from the structural schema rather than generic statistical pretraining. Together, these results provide a mechanistic account of how task demands shape low-rank recurrent dynamics to support relational abstraction and transfer in sequential computation.

1.1. Previous Work

Understanding how neural systems represent and generalize sequential structure has been a longstanding goal in both neuroscience and machine learning. Early work focused on mechanisms for maintaining and recalling of ordered information, especially in working memory, for instance by combining item and rank information (Botvinick & Watanabe, 2007). More recently, RNNs have been shown to learn generalized positional structures (Chu et al., 2023). Related approaches use chunking or segmentation to extract interpretable units from high-dimensional neural dynamics underlying concepts, including structural schema (Wu et al., 2025a).

Another line of work augments RNNs with external memory or plasticity mechanisms, allowing sequences to be represented as trajectories over latent states or “cognitive maps” (Whittington et al., 2025). While powerful, these models often rely on local or Markovian transitions, and their ability to generalize to non-adjacent or higher-order dependencies—critical for abstraction—remains unclear.

From a dynamical systems perspective, low-rank recurrent connectivity is known to confine neural activity to task-specific low-dimensional manifolds (Mastrogiuseppe & Ostojic, 2018; Schuessler et al., 2020). Our work builds on this framework by showing how low-rank recurrent dynamics give rise specifically to abstract relational representations and enable generalization and transfer. In contrast to prior studies focused on positional order or explicit memory architectures, we demonstrate that standard RNNs can spontaneously learn abstract relational patterns (e.g., AAB) through low-rank dynamics alone. Moreover, we establish a causal role for the dominant singular components in integrating transition history, identifying a concrete circuit-level mechanism by which recurrent networks internalize hierarchical

structure.

2. Results

Throughout this work, we consider different neural network models to solve different tasks. Each of these models has a standard discrete-time RNN as a core component. The inputs to the RNN, denoted $\mathbf{x} \in \mathbb{R}^\alpha$, are a one-hot encoding of the elements in the sequence over letters in the alphabet. The μ -th sequence, $\mathbf{X}^\mu$, with $\mu = 1 \dots p$, is given by the elements $\mathbf{x}_t^\mu$, for $t = 1 \dots L$. Upon processing this sequence, the hidden activity $\mathbf{h}^\mu \in \mathbb{R}^N$, evolves according to

$$\begin{aligned} \mathbf{z}_t^\mu &= \mathbf{W}_h \cdot \mathbf{h}_{t-1}^\mu + \mathbf{W}_{in} \cdot \mathbf{x}_t^\mu + \mathbf{b}_h, \\ \mathbf{h}_t^\mu &= \phi(\mathbf{z}_t^\mu). \end{aligned} \quad (1)$$

where $\mathbf{W}_h \in \mathbb{R}^{N \times N}$ is the recurrent weight matrix, $\mathbf{W}_{in} \in \mathbb{R}^{N \times \alpha}$ is input weight matrix, $\mathbf{b}_h \in \mathbb{R}^N$ are constant biases and ϕ is a non-linear function applied unit-wise. Here, we choose to work with rectified-linear units, i.e. $\phi(z) = \max\{0, z\}$. In the following, we will omit the sequence index unless necessary. All networks are trained via gradient-based optimization (Adam (Kingma & Ba, 2014)) using backpropagation through time and performing mini-batch updates. Different tasks require different targets and loss functions, and the models used for different tasks only differ in the readout (see SI Sec. A for more details).

2.1. RNNs can classify sequences with regularity across a wide range of parameters

We first examine a sequence classification task in which the network is trained to classify sequences based on their underlying temporal structure (see SI Sec. B for details on the generative process for constructing sequences). For example, sequences such as abbbb and addd are generated from the class ABBB, while abba and cddc belong to the class ABBA (Fig. 1A). The network receives a label only at the end of the sequence, and the loss function is the cross-entropy between the output class probabilities generated by the network and the ground truth (see SI Sec. A). In the remainder of the main manuscript, we focused on networks initialized in the “lazy” regime (Flesch et al., 2022), where initial weights prior to training are drawn from a uniform distribution with a standard deviation of $N_{\text{input}}^{-1/2}$; we have replicated all notable results in the “rich” regime in Fig. S5.

We find that sufficiently large networks achieve near zero loss on both training and test sets, indicating strong generalization performance (Fig. 1C and Fig. S1). Also, the number of classes that can be reliably generalized scales with network size, with larger networks required for generalizing a larger number of classes (Fig. S1, right).

In order to better understand the types of representations supporting generalization, we applied Principal Component Analysis (PCA) to the hidden activities at the end of the sequence. For sequences of length $L = 4$ and number of classes $C = 4$, the top three principal components account for $\sim 90\%$ of the variance, revealing a highly structured, low-dimensional geometry. Moreover, the hidden activities strongly cluster by class, and are linearly separable according to transition types: whether the sequence transitions to the same or a different letter (Fig. 1D, left). When we projected the hidden activities onto the same reduced space, as the sequences unfold, we observed a branching pattern that mirrors the hierarchical tree structure used to generate the sequences (Fig. 1D, right). These results suggest that the RNN constructs internal representations that compress and organize sequence information in a way that reflects the abstract structure of the task. Next, we investigate the geometry of these unfolding representations, and the recurrent mechanism that supports it.

2.1.1. LOW-DIMENSIONAL DYNAMICS REFLECTS LOW-RANK RECURRENT CONNECTIVITY

We begin by analyzing the dimensionality of the hidden activity in the classification-trained RNN as a function of time. To do so, we apply Singular Value Decomposition (SVD) to the hidden activities after each item in the sequence is presented.

We define the dimensionality of this hidden activity at each time step as the number of singular vectors required to capture at least 90% of the mean squared activity. We find that the hidden activity lies in a consistently lower dimensional space compared to the input space, and that dimensionality tends to decrease over time (Fig. 2A). This decrease in dimensionality is class-dependent: dimensionality increases with a transition to a different letter, but decreases towards the end of the sequence (Fig. 2A). This progressive dimensionality reduction suggests that the recurrent weight matrix $\mathbf{W}_h$ is effectively low-rank, consistent with (Mastrogiuseppe & Ostojic, 2018), up to a noisy term that reflects randomness in initial conditions and/or the training protocol (Fig. 2B)

$$\mathbf{W}_h = \sum_{\rho=1}^r S_{\rho} \mathbf{l}_{\rho} \mathbf{r}_{\rho}^{\top} + \delta \mathbf{W}_h. \quad (2)$$

If the geometry of the low-rank component is constant across simulations, i.e. the singular vectors $\mathbf{l}_{\rho}$ and $\mathbf{r}_{\rho}$ are the same up to a global transformation, then one should be able to isolate them by averaging over an ensemble of network initializations. To this end, we trained K networks with identical data and task objectives, but different random initializations and minibatch updates. We expect that any differences in learned weights across networks should be attributable to a global transformation. Since the data and

the task are identical across simulations, we expect that this transformation can be found by comparing the singular vectors of the $p \times N$ matrix $\mathbf{H}$, containing the hidden activity at the final time step upon readout, i.e. $H_i^{\mu} = h_{i,t=L}^{\mu}$.

For each experiment, denoted $k = 1 \dots K$, we apply SVD to the corresponding activity matrix $\mathbf{H}^{(k)}$. We denote $\mathbf{U}^{(k)}$ and $\mathbf{V}^{(k)}$ the matrices whose columns are the left and right singular vectors, respectively, and $\mathbf{S}^{(k)}$ the diagonal matrix with the corresponding singular values, so that $\mathbf{H}^{(k)} = \mathbf{U}^{(k)} \mathbf{S}^{(k)} \mathbf{V}^{(k)\top}$. We find that while the singular value spectra $\mathbf{S}^{(k)}$ are highly consistent across runs, the singular vectors themselves are randomly orientated. We can use the right singular vectors matrices $\mathbf{V}^{(k)}$ as the orthonormal reference frames, with respect to which the hidden activity can be compared across simulations. In this rotated frame, the hidden activity is written as

$$\bar{\mathbf{h}}^{(k)} = \mathbf{R}^{(k)} \cdot \mathbf{h}^{(k)}, \quad (3)$$

where $\mathbf{R}^{(k)} = \mathbf{V}^{(k)\top}$. Similarly, we can compare the learned weights across simulations via an analogous change of reference frame. In order for the currents $\mathbf{z}$ in Eq. 1 to transform in the same way as Eq. 3, i.e. $\bar{\mathbf{z}}^{(k)} = \mathbf{R}^{(k)} \cdot \mathbf{z}^{(k)}$, the recurrent weights must transform as

$$\bar{\mathbf{W}}_h^{(k)} = \mathbf{R}^{(k)} \mathbf{W}_h^{(k)} \mathbf{R}^{(k)\top}. \quad (4a)$$

and the input weights as

$$\bar{\mathbf{W}}_{in}^{(k)} = \mathbf{R}^{(k)} \mathbf{W}_{in}^{(k)}. \quad (4b)$$

Finally, in order for the outputs to remain unchanged, we need to transform the readout weights as

$$\bar{\mathbf{W}}_{out}^{(k)} = \mathbf{W}_{out}^{(k)} \mathbf{R}^{(k)\top}. \quad (4c)$$

In the case of a linear network, where ϕ is the identity, Eq. 1 are clearly invariant under rotations. If ϕ is the rectified-linear activation function, this is generally not the case.

However, we see that the symmetry holds approximately, as shown by the fact that the entries of $\bar{\mathbf{W}}_h$ and $\bar{\mathbf{W}}_{out}$ are approximately the same across simulations (Fig. 2C). Our working hypothesis is that, modulo a simulation-specific rotation, differences in the recurrent weight matrices across simulations are accounted for by the ‘‘quenched’’ noise term $\delta \mathbf{W}_h$ in Eq. 2, while the low-rank part, supporting task demands, remains the same (Fig. 2D). Therefore, we can estimate the rank of the structured part by comparing the spectrum (singular values) of the average transformed weight matrices, $\langle \bar{\mathbf{W}}_h \rangle = K^{-1} \sum_k \bar{\mathbf{W}}_h^{(k)}$, with that of the average original weights matrix, $\langle \mathbf{W}_h \rangle = K^{-1} \sum_k \mathbf{W}_h^{(k)}$. While the former retains the common low-rank structure, in the latter, the random relative orientation of the structured part is averaged away. We estimate the rank R of the common

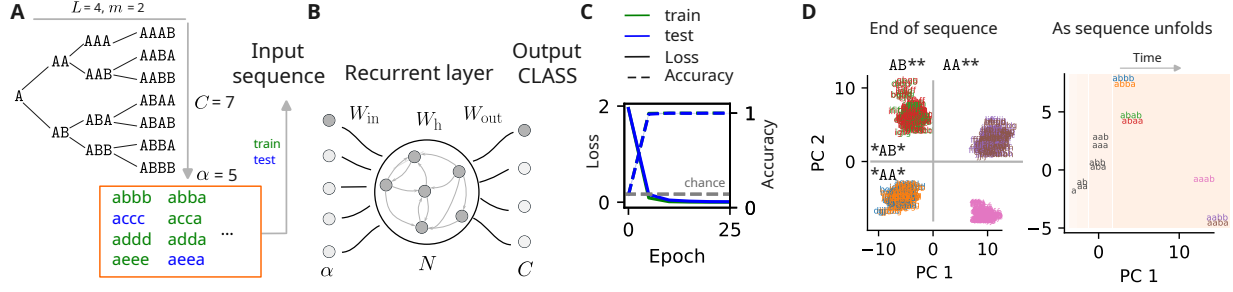


Figure 1. Emergence of abstract sequence representations in recurrent networks. **A.** Sequences are generated from a binary branching tree (example shown for sequence length $L = 4$, and $m = 2$ unique letters, corresponding to the number of branches from the root), yielding $C = 7$ abstract classes. Each class is instantiated by replacing the m unique letters with symbols drawn from an alphabet (of length $\alpha = 5$). **B.** An RNN processes sequences one token at a time and is trained to classify them according to their abstract structure, receiving a class label only at the end. **C.** Training loss and accuracy across epochs (mean $\pm$ s.d. across simulations). **D.** PCA of hidden activities at the end of training and end of sequence. Left: final hidden states cluster by abstract class and are linearly separable according to transition to same/different letter. Right: trajectories of hidden activity for one example sequence per class (indicated in different colors) plotted for each time-point as the sequence unfolds. PC2 and 3 capture branching dynamics that reflect the hierarchical structure of the generative tree as the sequence unfolds.

component to be the number of singular values of $\langle \bar{W}_h \rangle$ larger than those of $\langle W_h \rangle$. For the dataset presented here, $R = 3$, which corresponds to the number of singular values before the “elbow” in the spectrum of $\langle \bar{W}_h \rangle$ (Fig. 2D).

In the next section, we explore how the structure of the weights, i.e. the dominant singular vectors, relate to the statistical and relational structure of the input sequences.

2.2. Low-rank recurrent weights drive relational structure in the learned representations

The sequences used in this work are defined by temporal regularities described by a binary branching tree (Fig. 1A). A central question is how this relational structure is encoded in the RNN’s recurrent connectivity. We first examine the cosine similarity of hidden activity across sequences (Fig. 3A). As the sequence unfolds, the similarity matrix progressively reflects *only* the transition type: sequences continued by a “same” transition (AA-type) become more similar to each other than to those continued by a “different” transition (AB-type), independent of token identity. This emerges clearly at $t = 2$, where the similarity matrix separates in two blocks corresponding to AA and AB. At later timepoints, these diagonal blocks strengthen, and additional off-diagonal blocks appear, reflecting higher similarity among sequences sharing the *most recent* transition. By the classification time ($t = 4$), however, the matrix *primarily* reflects the *earliest* transitions: sequences sharing the initial branch of the tree exhibit more similar hidden activity than those diverging early.

Thus, the representations approximate a binary branching tree in which sequences are separated according to their transition structure in chronological order (Fig. 1A). To quantify the degree to which these representations exhibit hierarchical, tree-like geometry, we compute the *ultra-metric con-*

tent (UC) of the representations (Treves, 1997; Boboeva et al., 2018) at each time step (Fig. 3B-C). UC measures how closely a set of representations conforms to an ultrametric (tree-like) space (Fig. 3B, see SI D for details). We find that UC increases as the sequence unfolds (Fig. 3C, solid lines). The rise of UC beyond within-class baselines (dashed lines) reflects the increasing depth of the underlying tree. Importantly, this hierarchical structure is expressed in a compact, low-dimensional form: projecting the hidden activity onto the top singular vectors of the recurrent weight matrix W_h recovers most of the ultrametric structure observed in the full representation (Fig. 3C).

We note that the tree in Fig. 1A and 3B is not the only tree consistent with the representational geometry. One can construct a *backward* tree, starting from the last transition and branching in *reverse* order, which yields a different arrangement of the leaf classes. The cosine similarity matrix reflects contributions from *both* hierarchies: strong diagonal blocks correspond to early (forward) transitions, while the weaker off-diagonal blocks arise from shared *latest* transitions (backward hierarchy). Because the network processes sequences chronologically, the forward hierarchy is more salient, but the presence of structured off-diagonal similarity shows that the model retains information about late transitions as well. Thus, these deviations do not reflect noise; they reveal that the network’s representations interpolate between the forward and backward relational structures inherent in the task.

Last, we asked whether the quality of the abstract representations, as measured by the UC score, correlate with the network’s ability to generalize. For increasing number of classes, we computed both the mean UC score at the end of learning and the corresponding generalization accuracy. For networks of appropriate size, we observe a positive

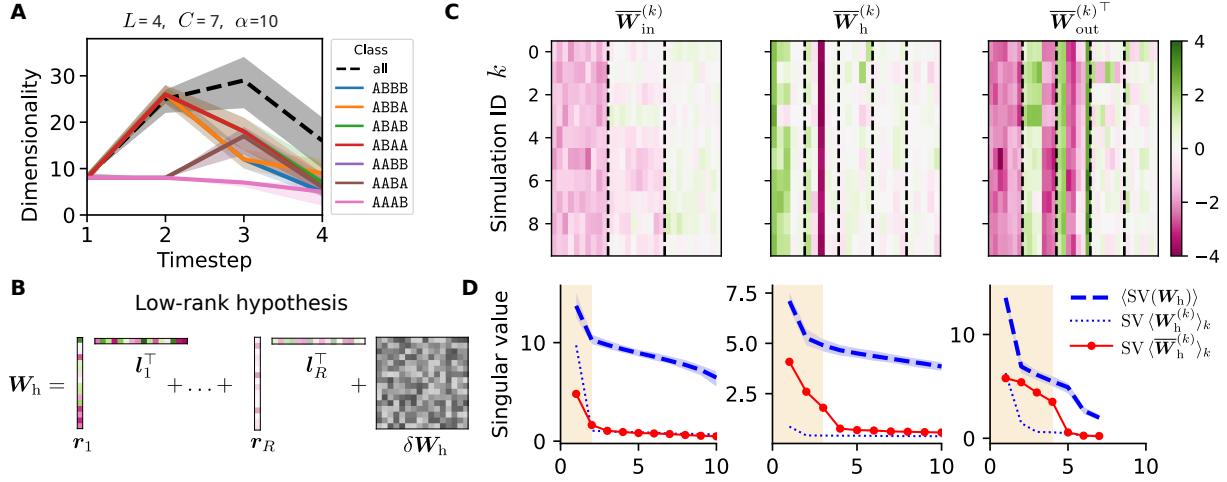


Figure 2. Low-dimensional dynamics and low-rank connectivity in trained recurrent networks. **A.** Dimensionality of hidden activity over time, measured as the number of singular vectors capturing 90% of the mean squared activity (medians $\pm$ 5th-95th percentiles). Dimensionality is class-dependent; typically rising mid-sequence and decreasing toward the end. **B.** Recurrent connectivity matrix shown as a sum of a structured low-rank component (colored) and a random component (gray). **C.** Input, recurrent and output weight matrices from multiple simulations shown in a common rotated coordinate frame (see Eqs. 4). Each row corresponds to one simulation; matrices are flattened by concatenating rows (input/recurrent) or columns (output transpose). Only the dominant modes are shown (3 for input (left), 5 by 5 for recurrent (middle) and 5 for output (right) weights). Note the consistent structure across simulations. **D.** Low-rank structure extracted by averaging aligned weight matrices across simulations. Spectra of recurrent matrices are similar across simulations (dashed blue line: mean of SVs of the weight matrices from individual simulations $\pm$ 5 – 95% percentiles bands). Spectra of the average recurrent weights before and after change of basis (dotted blue vs red) differ only in their structured components (shaded red region, used to infer the rank of the structured part). Left: spectrum of input weights. Middle: recurrent where $R = 3$. Right: output.

correlation: networks that settle into more strongly hierarchical representations generalize better, suggesting that the quality of the internal abstract configuration directly drives generalization (Fig. 3E)).

2.3. Perturbation experiments reveal functional role of dominant singular components

Next, we asked whether the low-rank component of the recurrent weights is sufficient to support the classification task, and what computational role the dominant singular components play in shaping abstract representations.

While averaging weight matrices across aligned networks reveals the approximate singular values $\{S_\rho\}$ of the low-rank part, the corresponding singular vectors cannot be cleanly extracted: as seen in Fig. 2D (middle panel), the low-rank structure and the noise components of W_h have comparable magnitudes (as seen by comparing the dashed blue line with solid red line, and the gap between them), so the singular vectors of the full matrix do not reliably isolate the underlying low-rank subspace. Nevertheless, we can still test functional sufficiency by training explicitly rank-constrained networks.

We parametrized the recurrent weights matrix as a product $W_h = AB^\top$, where A and B are $\mathbb{R}^{N \times r}$ trainable matrices, and varied the maximum rank r . Networks of rank 3 (as

well as 4 and 5) learn more slowly than full-rank models but ultimately match their memorization performance and achieve nearly identical generalization (Fig. 4A). In contrast, rank 1 and rank 2 networks show marked deficits; their confusion patterns reflect their inability to distinguish abstract classes that are close in representational space (Fig. 4B-C).

To interpret the functional role of the singular components, we performed *singular vector ablation* on trained networks with recurrent weights W_h . After training a rank-constrained network, we removed selected singular vectors from the recurrent weights, then simulated the network on all sequences, and computed its performance. The perturbed recurrent weight matrix where the a -th singular component has been removed would be $\tilde{W}_h^{(a)} = W_h - s_a l_a r_a^\top$, such that that $\tilde{W}_h^{(a)} r_a = 0$ i.e. any activity along r_a is not propagated to the next step.

Examining hidden activity projected onto the dominant singular vector reveals its functional meaning. At $t = 2$, projections along the first singular vector split cleanly into two blocks reflecting same/different transitions (AA/BB vs. AB/BA, Fig. S4). At subsequent timesteps, each block subdivides according to the *most recent* transition. By $t = 4$, this yields perfect decoding of abstract classes: the projection carries integrated information about the full transition history, hence the hierarchical generative tree (Fig. 4D left

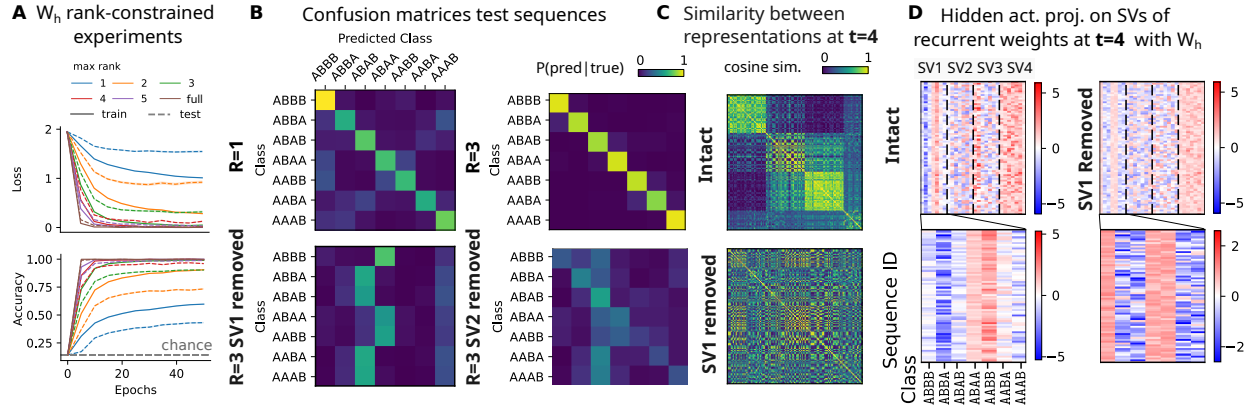


Figure 4. Perturbation of singular components reveals how low-rank dynamics support abstraction. **A.** We train networks with rank-constrained connectivity (ranks 1 – 5) and find that networks with low-rank recurrent connectivity can learn the task. **B.** Confusion matrices reveal the nature of errors. Perturbing a trained rank-3 recurrent matrix by removing individual singular components (bottom) show that eliminating the leading component (SV1) causes sequences to be classified almost solely by their last transition, “same” vs “different” (i.e. AA vs AB, collapsing many classes (bottom left). Removing SV2 yields milder distortions (bottom right). **C.** Cosine similarity matrices show that the structured clustering present in the intact network is disrupted when SV1 is removed. **D.** Projections of hidden activity onto the dominant (right) singular vectors illustrate the perturbation’s effect: with SV1 removed, the projection along that direction reflects only the most recent transition, while other projections carry little class information. In the intact network, SV1 (and to a lesser extent SV2-SV3) integrate transition information over time, supporting the formation of hierarchical, class-specific representations (see Fig. S4 for full time courses).

tional structure underlying the sequences.

A key challenge for training the prediction network was constructing unambiguous sequences. In classification, the network receives full sequences; in prediction, the network is cued with a partial sequence and must recursively generate the continuation, feeding each predicted item back as input.

Using the same sequences as in classification proved problematic: many cues were ambiguous, producing multiple valid continuations, and the network defaulted to memorizing training examples rather than generalizing. To resolve this, we generated cues from the binary tree structure and then pruned the tree, retaining only a single valid continuation for each cue (Fig. 5A). This ensured that each prediction target was uniquely defined.

Using this protocol, prediction networks trained on the same pruned sequence distribution as classification networks (Fig. 5B) maintained strong memory capacity (Fig. S2, top) but generalized more poorly (Fig. S2, bottom). Their internal dynamics also differed markedly: in contrast to classification networks, dimensionality remained high throughout the sequence (Fig. 5D, green), the recurrent weights lacked low-rank structure (Fig. 5E), and their representations showed weaker hierarchical organization (Fig. 5F). Thus, low-rank dynamics do not arise merely from processing the same input statistics; they appear specific to tasks requiring global integration across the full sequence (Turner & Barak, 2024; Maheswaranathan et al., 2019).

Next, we examined whether transferring the weights from a classification-trained model could accelerate convergence

and improve generalization in the prediction task. We transferred recurrent and input weights (and biases) from a classification-trained model and tested several regimes: freezing the transferred weights (except outputs) (Fig. 5E, red), or using them as initialization for continued training (Fig. 5E, orange). Freezing the weights offered no benefit over chance (Fig. 5E). In contrast, jointly initializing both input and recurrent weights from the classification task led to faster learning of the test set and improved asymptotic generalization performance (Fig. 5E). We find that this benefit is mostly due to transferring the recurrent weights and not the input weights (Fig. S3A, compare purple with cyan).

Finally, to test whether these improvements are specifically due to the abstract scaffold and not generic pretraining, we trained an autoencoder model to reconstruct sequences (Fig. S3E). Transferring encoder weights to the prediction model sped up memorization but did not improve generalization, unlike classification pretraining. This dissociation indicates that the transfer benefit arises from the learned abstract relational scaffold, not from shared data statistics or generic pretraining.

4. Conclusion and discussion

We showed that abstract temporal structure can emerge spontaneously in recurrent neural networks trained only on sequence-level labels. Without architectural biases or intermediate supervision, standard RNNs develop compact, low-dimensional internal representations that reflect the temporal hierarchical structure of the input sequences and gen-

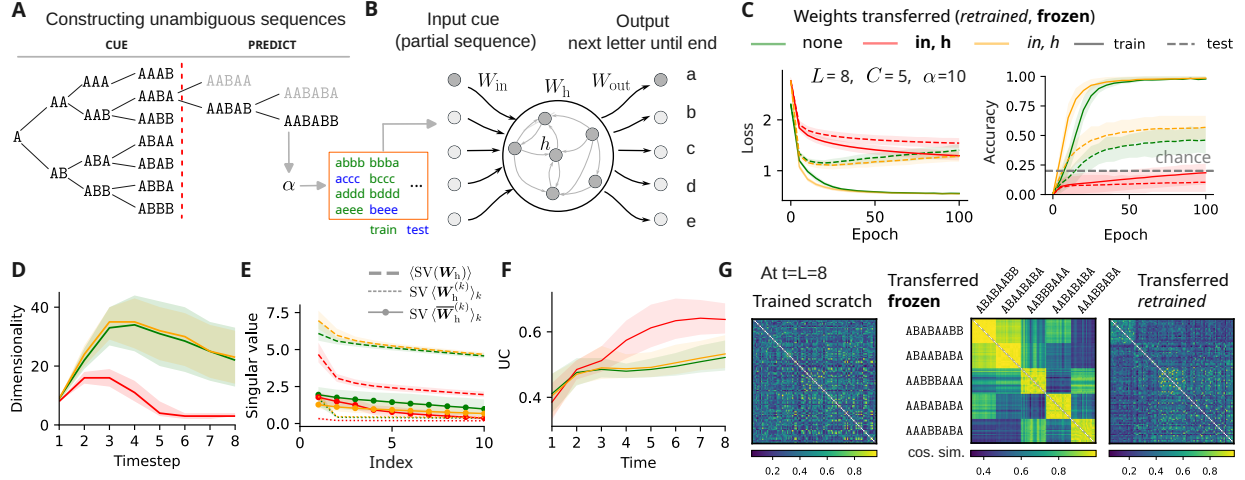


Figure 5. Transfer from classification improves prediction learning. **A.** To construct unambiguous sequences for the prediction task, we apply extreme pruning to a binary tree (light gray), retaining only one path per cue (black). The red dashed line marks the boundary between the cue (input) and the continuation to be predicted. Unique tokens are then replaced by letters drawn from an alphabet of size α . **B.** Left: a standard RNN is trained from scratch to predict each next token in the continuation sequence. Right: prediction networks are either initialized with weights transferred from a classification-trained RNN and then retrained (italic labels), or evaluated with transferred recurrent weights kept fixed (bold). **C.** Loss (left) and accuracy (right) for prediction networks under three initializations: random (green), transferred recurrent weights kept frozen (red; only output weights trained), and transferred recurrent weights used as initialization and then jointly trained (orange). Means and standard deviations shown across simulations. Only the latter condition accelerates memorization and improves generalization. To reduce overfitting, training was stopped at epoch 25 in all cases. **D.** Dimensionality of hidden activity over time, measured as the number of singular vectors capturing 90% of the mean squared activity (Median $\pm 5 - 95\%$ percentile bands). Dimensionality decreases mainly when transferred recurrent weights are frozen (red); networks trained from scratch (green) or fine-tuned from classification (orange) maintain higher dimensionality. **E.** Spectra of recurrent matrices (averaged over simulations, dashed), and spectra of their averages before and after change of basis (dotted vs solid). Only frozen transferred weights (red) yield a clear structured low-rank component ($R = 3$). Prediction networks trained from scratch (green) show no low-rank structure; those initialized from classification (orange) also lack clear low-rank structure, though their spectrum is less flat than randomly initialized networks. **F.** Ultrametric content (UC) of hidden representations over time for each initialization condition (as in Fig. 3C, mean UC $\pm 5 - 95\%$ percentile bands). **G.** Cosine similarity matrices between hidden representations for each initialization condition. Left: prediction net trained from scratch. Middle: weights transferred from classification to prediction and frozen. Right: weights transferred from classification and then retrained. Only the transfer-and-freeze condition preserves the hierarchical, low-dimensional representations.

eralize across token identities. This abstraction is supported by low-rank recurrent connectivity, which constrains neural dynamics into a shared low-dimensional manifold aligned with the generative tree.

By combining rank-constrained training with targeted singular component perturbations, we identified a causal role for the dominant recurrent mode: it integrates past and current same/different transitions at each timestep, enabling the progressive separation of classes along the generative tree as the sequence unfolds. Removing this component selectively erases memory for earlier transitions, collapsing the tree-like structure and revealing the mechanistic function of the leading singular direction.

Crucially, we showed that this learned low-rank scaffold is reusable: transferring recurrent weights from classification to a next-token prediction task improves generalization, whereas generic pretraining (e.g. autoencoder) does not. Together, these findings suggest that even simple recurrent architectures can internalize latent statistical structure in a flexible, and reusable way, offering a potential compu-

tational mechanism for abstraction in biological circuits. The resulting low-dimensional geometry also yields testable predictions for neural population activity, which we are currently investigating in electrophysiological recordings from rodents learning structurally analogous sequences, testing whether hippocampal and frontal populations exhibit similar low-dimensional, transition-integrating dynamics.

Several limitations remain. Our study focuses on algebraic patterns generated by deterministic binary trees and fixed sequence lengths; whether similar mechanisms extend to probabilistic grammars, naturalistic statistics, or variable-length sequences remains open. Real-world environments also involve noise, stochastic transitions, hierarchical dependencies at multiple timescales, and multiple interacting objectives; these may reshape how abstract representations form. Finally, we analyzed only end-of-training representations; understanding how low-rank structure and abstraction emerge over learning (Saxe et al., 2019) is an important direction for future work.

REPRODUCIBILITY STATEMENT

This manuscript provides sufficient detail to reproduce the main experiments, including architectures, training procedures, and sequence generation. All key parameters are specified, with full configurations in the Supplementary Material. Sequences are synthetically generated using a deterministic procedure described in the Methods. Code for sequence generation and model training is available at https://anonymous.4open.science/r/ICML2026_submission-E774/, and analysis code will be released upon publication.

IMPACT STATEMENT

This work complies with the ICML Code of Ethics. All data are synthetically generated and involve no real-world individuals, environments, or sensitive information. The study poses no foreseeable safety, security, or privacy risks and does not relate to applications involving deception, discrimination, surveillance, or weaponization. Computational demands are modest, with lightweight models trained on a limited compute budget, resulting in minimal environmental impact.

References

- Barak, O. Recurrent neural networks as versatile tools of neuroscience research. *Current Opinion in Neurobiology*, 46:1–6, 2017.
- Boboeva, V., Brasselet, R., and Treves, A. The capacity for correlated semantic memories in the cortex. *Entropy*, 20 (11):824, 2018.
- Botvinick, M. and Watanabe, T. From numerosity to ordinal rank: a gain-field model of serial order representation in cortical working memory. *Journal of Neuroscience*, 27 (32):8636–8642, 2007.
- Chu, Z., Guo, Q., Cheng, J., Ho, B., Wu, S., Mi, Y., et al. Learning and processing the ordinal information of temporal sequences in recurrent neural circuits. *Advances in Neural Information Processing Systems*, 36:33999–34020, 2023.
- Dehaene, S., Meyniel, F., Wacongne, C., Wang, L., and Pallier, C. The neural representation of sequences: from transition probabilities to algebraic patterns and linguistic trees. *Neuron*, 88(1):2–19, 2015.
- Driscoll, L., Shenoy, K., and Sussillo, D. Flexible multi-task computation in recurrent networks utilizes shared dynamical motifs. *bioRxiv*, pp. 2022–08, 2022.
- Flesch, T., Juechems, K., Dumbalska, T., Saxe, A., and Summerfield, C. Orthogonal representations for robust context-dependent task performance in brains and neural networks. *Neuron*, 110(7):1258–1270, 2022.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Machens, C. K., Romo, R., and Brody, C. D. Flexible control of mutual inhibition: a neural model of two-interval discrimination. *Science*, 307(5712):1121–1124, 2005.
- Maheswaranathan, N., Williams, A., Golub, M., Ganguli, S., and Sussillo, D. Universality and individuality in neural dynamics across large populations of recurrent networks. *Advances in neural information processing systems*, 32, 2019.
- Marcus, G. F. *The algebraic mind: Integrating connectionism and cognitive science*. MIT press, 2003.
- Mastrogiuseppe, F. and Ostojic, S. Linking connectivity, dynamics, and computations in low-rank recurrent neural networks. *Neuron*, 99(3):609–623, 2018.
- Mikolasch, S., Kotrschal, K., and Schloegl, C. Transitive inference in jackdaws (*corvus monedula*). *Behavioural processes*, 92:113–117, 2013.
- Murphy, R. A., Mondragón, E., and Murphy, V. A. Rule learning by rats. *Science*, 319(5871):1849–1851, 2008.
- Santolin, C. and Saffran, J. R. Constraints on statistical learning across species. *Trends in Cognitive Sciences*, 22 (1):52–63, 2018.
- Santolin, C., Rosa-Salva, O., Regolin, L., and Vallortigara, G. Generalization of visual regularities in newly hatched chicks (*gallus gallus*). *Animal Cognition*, 19:1007–1017, 2016.
- Saxe, A. M., McClelland, J. L., and Ganguli, S. A mathematical theory of semantic development in deep neural networks. *Proceedings of the National Academy of Sciences*, 116(23):11537–11546, 2019.
- Schuessler, F., Dubreuil, A., Mastrogiuseppe, F., Ostojic, S., and Barak, O. Dynamics of random recurrent networks with correlated low-rank structure. *Physical Review Research*, 2(1):013111, 2020.
- Spierings, M. J. and Ten Cate, C. Budgerigars and zebra finches differ in how they generalize in an artificial grammar learning experiment. *Proceedings of the National Academy of Sciences*, 113(27):E3977–E3984, 2016.
- Treves, A. On the perceptual structure of face space. *BioSystems*, 40(1-2):189–196, 1997.

Supplementary Information

- Turner, E. and Barak, O. The simplicity bias in multi-task
rnns: Shared attractors, reuse of dynamics, and geometric
representation. *Advances in Neural Information Process-
ing Systems*, 36, 2024.
- Urcelay, G. P. and Miller, R. R. On the generality and limits
of abstraction in rats and humans. *Animal Cognition*, 13:
21–32, 2010.
- Wang, L., Uhrig, L., Jarraya, B., and Dehaene, S. Represen-
tation of numerical and sequential patterns in macaque
and human brains. *Current Biology*, 25(15):1966–1974,
2015.
- Whittington, J. C., Dorrell, W., Behrens, T. E., Ganguli, S.,
and El-Gaby, M. A tale of two algorithms: Structured
slots explain prefrontal sequence memory and are unified
with hippocampal cognitive maps. *Neuron*, 113(2):321–
333, 2025.
- Wu, S., Alaniz, S., Karthik, S., Dayan, P., Schulz, E., and
Akata, Z. Concept-guided interpretability via neural
chunking. *arXiv preprint arXiv:2505.11576*, 2025a.
- Wu, S., Thalmann, M., and Schulz, E. Two types of mo-
tifs enhance human recall and generalization of long se-
quences. *Communications Psychology*, 3(1):3, 2025b.
- Yang, G. R. and Molano-Mazón, M. Towards the next
generation of recurrent network models for cognitive neu-
roscience. *Current Opinion in Neurobiology*, 70:182–192,
2021.

A. Neural Network Models

Classification model In this task, the output of the RNN model is the probability distribution $\mathbf{p}$ over C classes, therefore in the $(C - 1)$ -dimensional simplex Δ^{C-1} . We choose to minimize a cross-entropy (CE) function

$$l(X, y) = - \sum_{c=1}^C y^c \log p^c(X), \quad (S1)$$

where y^c is the target probability that the sequence X belongs to class c (1 if c equals the true class, 0 otherwise). The output is computed after the L -th (last) element in the sequence X :

$$\mathbf{p} = \text{softmax}(\mathbf{W}_{\text{out}} \cdot \mathbf{h}_L + \mathbf{b}_{\text{out}}) \quad (S2)$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{C \times N}$ is the output weight matrix, i.e.

$$p^c = \frac{e^{\mathbf{w}_{\text{out}}^c \cdot \mathbf{h}_L + b_{\text{out}}^c}}{\sum_j e^{\mathbf{w}_{\text{out}}^j \cdot \mathbf{h}_L + b_{\text{out}}^j}}. \quad (S3)$$

where $\mathbf{w}_{\text{out}}^c$ and b_{out}^c are the c -th row of $\mathbf{W}_{\text{out}}$ and the c -th component of $\mathbf{b}_{\text{out}}$, respectively.

Prediction model In the prediction task, the output is a probability distribution over the letter in the alphabet, $\mathbf{p} \in \Delta^{\alpha-1}$. Here the objective is to predict the next letter in the sequence. Given a starting cue letter, the network outputs the next letter, and this next letter then serves as a cue for the prediction of the next letter. Therefore the loss function that we minimize is the cross entropy between sequence elements at time t and those at time $t + 1$.

$$l(\mathbf{X}, \mathbf{Y}) = - \sum_{t=1}^{L-1} \sum_{i=1}^{\alpha} Y_{t+1}^i \log p_t^i, \quad (S4)$$

where Y_{t+1}^i is the next letter in the sequence, and p_t^i is probability distribution of the next letter as predicted by the network, given by

$$\mathbf{p}_t = \text{softmax}(\mathbf{W}_{\text{out}} \cdot \mathbf{h}_t + \mathbf{b}_{\text{out}}). \quad (S5)$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{\alpha \times N}$.

Reconstruction model For this task, we consider an autoencoder architecture composed of an RNN encoder and an RNN decoder, with the same number of recurrent units, N .

At the end of the input sequence, a fully connected linear layer maps the recurrent activity of the encoder to an n -dimensional latent space.

The latent activity is then fed as a constant input to the decoder RNN through a linear. All the weights in the network are updated via batch gradient descent through BPTT.

$$l(\mathbf{X}, \tilde{\mathbf{X}}) = - \sum_{t=1}^{L-1} \sum_{i=1}^{\alpha} x_t^i \log \tilde{x}_t^i, \quad (S6)$$

where x_t^i is the input sequence (and the target), $\tilde{x}_t^i$ is the reconstructed sequence, given by

$$\tilde{x}_t^i = \text{softmax}(\tilde{\mathbf{W}}_{\text{out}} \cdot \tilde{\mathbf{h}}_t + \tilde{\mathbf{b}}_{\text{out}}) \quad (S7)$$

$$\tilde{\mathbf{h}}_t = \phi(\tilde{\mathbf{W}}_{\text{h}} \cdot \tilde{\mathbf{h}}_{t-1} + \tilde{\mathbf{W}}_{\text{lat}} \cdot \mathbf{z} + \tilde{\mathbf{b}}_{\text{h}}) \quad (S8)$$

$$\mathbf{z} = \mathbf{W}_{\text{lat}} \cdot \mathbf{h}_L + \mathbf{b}_{\text{lat}} \quad (S9)$$

$$\mathbf{h}_t = \phi(\mathbf{W}_{\text{h}} \cdot \mathbf{h}_{t-1} + \mathbf{W}_{\text{lat}} \cdot \mathbf{x}_t + \mathbf{b}_{\text{h}}) \quad (S10)$$

with $\mathbf{h}_0 = \mathbf{0}$ and $\tilde{\mathbf{h}}_0 = \mathbf{0}$ and where $\mathbf{h}, \tilde{\mathbf{h}}, \mathbf{b}_{\text{h}}, \tilde{\mathbf{b}}_{\text{h}} \in \mathbb{R}^N$, $\mathbf{x}_t, \tilde{\mathbf{x}}_t, \tilde{\mathbf{b}}_{\text{out}} \in \mathbb{R}^{\alpha}$, $\tilde{\mathbf{W}}_{\text{out}} \in \mathbb{R}^{\alpha \times N}$, $\tilde{\mathbf{W}}_{\text{lat}} \in \mathbb{R}^{N \times n}$, $\mathbf{b}_{\text{lat}}, \mathbf{z} \in \mathbb{R}^n$, and $\mathbf{W}_{\text{lat}} \in \mathbb{R}^{n \times N}$.

B. Generating structured sequences

We construct sequences designed to exhibit specific regularities. Two key parameters define the sequence space: the sequence length L , and the number of distinct letters m used in each sequence, with $m < L$, to ensure repetition and thus induce “regularity” in the sequence. For example, consider $L = 2$ and $m = 2$: the only possible sequence is AB . With $L = 3$ and $m = 2$, possible patterns include AAB , ABB and ABA , each exhibiting a different structure of repetition and alternation—e.g. “same different” vs “different same”. These patterns define what we refer to as *algebraic patterns* (Marcus, 2003), or abstract temporal schemas. The number of such abstract patterns correspond to the number of ways to partition a sequence of L items into groups of at least one element is given by

$$\sum_{n_1, n_2, \dots, n_m} \frac{L!}{n_1! n_2! \dots n_m!}, \quad (S11)$$

where the sum over $n_1, \dots, n_m$ is over partitions of L , i.e. they satisfy the constraint $\sum_{i=1}^m n_i = L$.

To instantiate each abstract pattern with concrete sequences, or tokens, we select m distinct letters from a given alphabet of size α . The number of such choices is

$$C(\alpha, m) = \frac{\alpha!}{(\alpha - m)! m!}. \quad (S12)$$

Thus, we produce p instances of the same sequence type, given by the multiplication of the two expressions above

$$p = \frac{\alpha!}{(\alpha - m)! m!} \sum_{n_1, n_2, \dots, n_m} \frac{L!}{n_1! n_2! n_m!} \quad (S13)$$

These sequences constitute a fraction of the total number of possible sequence configurations of length L given by

α^L . We use these structured sequences to train our model. In this manuscript, we used 80% of the sequences (chosen randomly) to train the models, and the remaining 20% to test the models. Choosing sequences randomly meant that networks were highly likely to have seen all the letters used in the data, but never in the configurations present in the test data.

Sequences produced using a binary branching tree In the special case of $m = 2$, this generative procedure defines a binary branching tree: each additional token in the sequence corresponds to a binary split (of an existing node), producing a hierarchical structure over the abstract classes (Fig. 1A, main text). Throughout the main text, we focus on this case to study how networks learn and represent such tree-structured temporal regularities.

Generating unambiguous sequences for testing generalization in prediction networks As mentioned in the main text, sequences generated from terminal nodes of the full binary branching tree naturally contain overlapping elements across classes. This overlap poses a challenge for evaluating generalization in the prediction task, which requires prompting the network with a partial sequence (of length $L_{cue} < L$), and having it recursively predict the remaining $L_{retr} = L - L_{cue}$. This meant that a partial cue could theoretically lead to multiple different retrieval options, leading to ambiguous sequences. When ambiguous cues were used, the prediction network often defaulted to generating familiar training-set continuations, and was unlikely to retrieve sequences in the test set. This made it difficult to assess its ability to generalize to unseen sequences. To address this, we designed a procedure to construct unambiguous classes for testing prediction performance. Specifically, we generated cues of length L_{cue} using the full tree structure, and then appended unique combinations of m letters beyond that point, effectively pruning all but one continuation path from the node L_{cue} onwards. Conceptually, this amounts to constructing a tree of depth L , from which all-but-one branches from node L_{cue} to the terminal nodes L of the tree are pruned (Fig. 4A, main text). This procedure meant that for a specific choice of L_{cue} and L , many different combinations of classes are possible, from which we sample C at a time (Fig. 4A of main text, black sequences sampled, gray sequences not sampled). To make sure that any effect that we observe is not due to a specific choice of this “class combination (CC)”, we run many different simulations with different samplings, with a maximum of $CC = 20$ unique class combinations, when available.

C. Phase diagrams

To generate the phase diagrams for both the classification network (Fig. S1) and the prediction network (Fig. S2) where we vary the number of classes C and the length of the sequences L , we used the pruning procedure to construct unambiguous sequences. This also allowed us to run transfer experiments (Fig. S3).

D. Ultrametric content

Given a set of p representations $\{\mathbf{h}^\mu\}_{\mu=1}^p$, we first compute the distance between any of the pairs $\mathbf{h}^\mu, \mathbf{h}^\nu$. Fig. 3B of main text shows a graphical depiction of sequences constructed using the binary branching process. Within this tree, the distance between any two terminal nodes (shown in color) can be defined as the distance to the nearest common ancestor. In order to compute the distance between pairs of real-valued neural representations, we simply compute the Euclidean distance between vectors of neural activity at any time-point (Fig. 3C, main text, top). We then take triplets of representations, $(\mathbf{h}^\mu, \mathbf{h}^\nu, \mathbf{h}^\rho)$. Denoting by $d_{\min}^{\mu\nu\rho}$ the edge of minimal length in the corresponding triangle, $d_{\max}^{\mu\nu\rho}$ the edge of maximal length and $d_{\text{med}}^{\mu\nu\rho}$ the edge of intermediate length, the ultrametric content (Treves, 1997) is defined as

$$\text{UC} = \frac{1}{N_{\text{tri}}} \sum_{1 \leq \mu < \nu < \rho \leq p} \frac{\log \delta_{\min}^{\mu\nu\rho} - \log \delta_{\text{med}}^{\mu\nu\rho}}{\log \delta_{\min}^{\mu\nu\rho} + \log \delta_{\text{med}}^{\mu\nu\rho}}, \quad (\text{S14})$$

where $\delta_{\min} = d_{\min}/d_{\max}$ and $\delta_{\text{med}} = d_{\text{med}}/d_{\max}$.

In Fig. 3C of main text, we compute the UC for the set of representations corresponding to all sequences (hidden activity at any given time-point, solid lines). As a control, to verify that the tree structure emerges due to the configuration of classes relative to one another, we also compute the UC at the end of the sequences, restricted to triplets belonging to the same class. It can be seen that towards the end of the sequence, as the class membership becomes apparent, the UC computed over all representations is above the UC values computed for triplets restricted to individual classes (0.5, Fig. 3C, main text).

If we plot $\delta_{\min} = f(\delta_{\text{med}})$, triplets that satisfy the triangular inequality lie above the line $\delta_{\min} = 1 - \delta_{\text{med}}$, while triplets that satisfy the ultra-metric inequality lie on the vertical line where $\delta_{\text{med}} = 1$. Triplets that are equilateral triangles lie at the point $\delta_{\min} = \delta_{\text{med}} = 1$.

Eq. S14 gives a measure of the overall closeness of the cloud of triplets to the fully ultra-metric limit. This quantity ranges from 0 (all triplets forming isosceles triangles with two short sides) to 1 (for a fully ultra-metric set: equilateral triangles and isosceles triangles with two long sides).

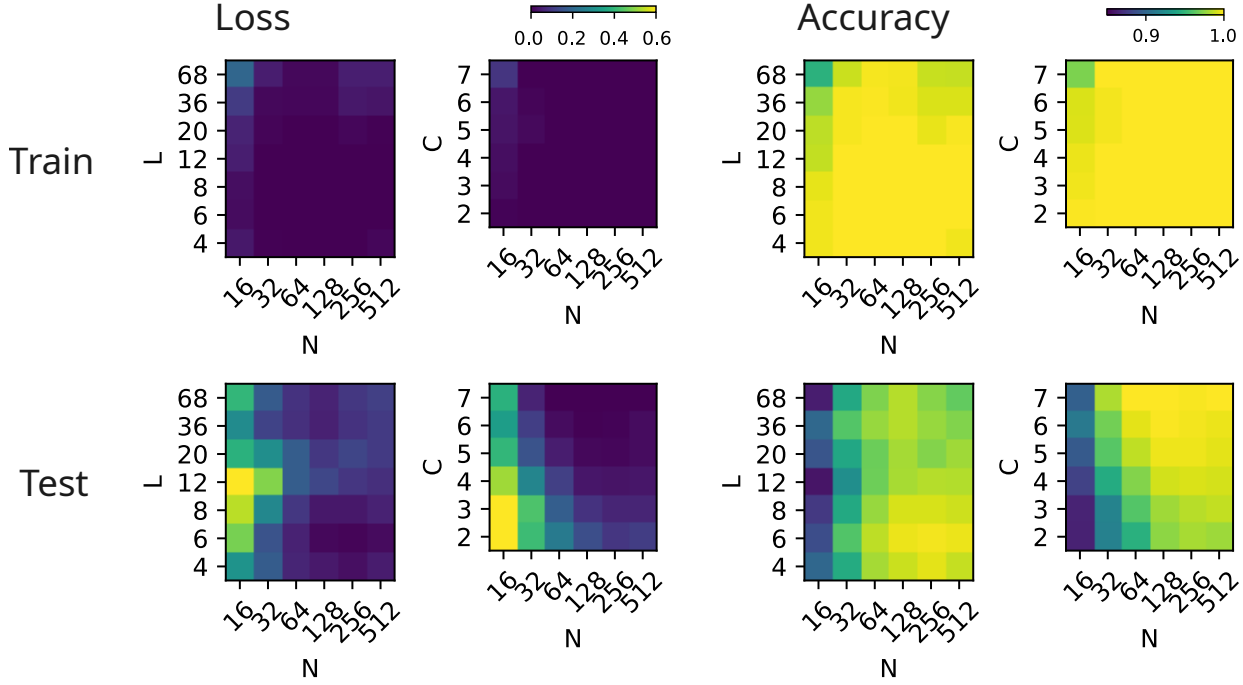


Figure S1. **Classification Task.** Phase diagrams showing train/test loss and accuracy as functions of network size N (x-axis) and either sequence length L or number of classes C (y-axes). When L is varied, the number of classes is fixed to $C = 4$. When C is varied, the sequence length is fixed to $L = 7$.

E. Transfer experiments

We studied three tasks using an RNN: one trained to classify sequences based on the abstract temporal structure of said sequences, a second trained to predict the next item in a sequence, and a third trained to reconstruct sequences.

Transferring classification pretrained weights to prediction network. While both classification and prediction trained networks exhibit good memory capacity as shown by their performance on sequences used in the training set, only the classification-trained network generalizes well across a wide range of parameters, particularly for longer sequences (Figs. S1 and S2). We found that the classification network achieves this via a progressive dimensionality reduction in its dynamics as the sequence unfolds (Fig. 2A, main text), which we linked to a low-rank structure in the recurrent connectivity matrix. In contrast, despite relying on similar sequence statistics (i.e. temporal patterns present in the classes), the prediction-trained network does not exhibit low-rank recurrent structure (Fig. 5D, main text). We note that one-step transition probabilities are not enough to perform the prediction task, and at any given time, the network requires knowledge of both the class and letter to perform this task. We hypothesized that the abstract structure learned by the classification network could be leveraged to improve prediction. To test this, we implemented multiple transfer

learning protocols: either 1) initializing the prediction network's weights and biases from a trained classifier and then freezing them (i.e. no more learning), or 2) initializing from the classifier and training afterwards. We applied these protocols separately to the input weights, recurrent weights, or both (Figs. S3A). When all parameters were frozen (protocol 1, red curve), prediction performance remained at chance (Fig. S3A). Freezing only the recurrent weights (blue curve) allowed for learning of memorization but not generalization. Freezing only input weights (pink) permitted learning but generalization was slow relative to learning from scratch (green).

Interestingly, when recurrent and input weights were transferred separately following protocol 2 (cyan and purple), we observed modest improvements in both memorization and generalization (Fig. S3A). However, combining both (input + recurrent; orange) led to sizeable gains in learning speed and generalization. To evaluate the robustness of this transfer, we ran this transfer protocol across a wide range of parameters, recurrent layer size N , number of classes C and sequence length L (Figs. S3B and C). Comparing against networks trained from scratch, we found that the transfer protocol 2 (input + recurrent) improved performance (in both memorization and generalization) early (epoch 10) and late (epoch 100) in training (green areas) for large-enough networks, but with no to negative effects in the performance

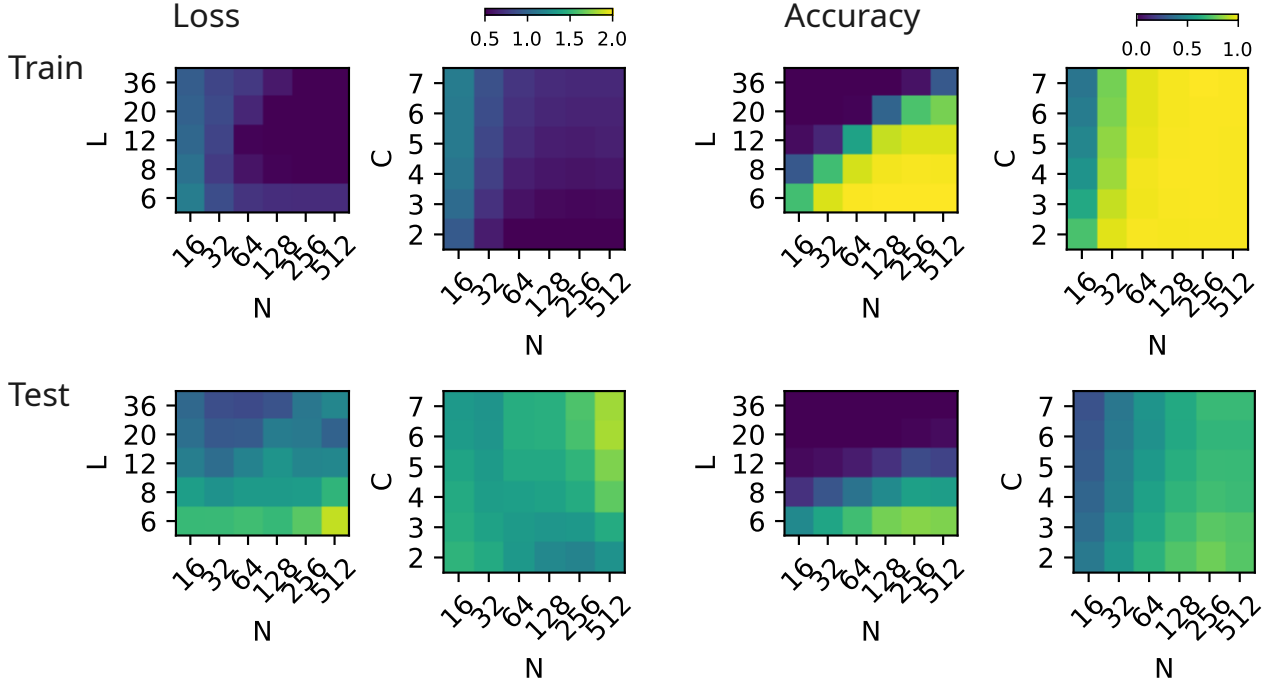


Figure S2. **Prediction Task.** Phase diagrams showing train/test loss and accuracy as functions of network size N (x-axis) and either sequence length L or number of classes C (y-axes). Networks reliably memorize training sequences, but show poor generalization to held-out sequences. Longer sequences and more classes require larger networks.

of very small networks (red areas).

Transferring reconstruction pretrained weights to prediction network. To control that any improvement in generalization performance when transferring classification-pretrained weights to the prediction network is due to the abstract scaffold and not due to a “warm start” hypothesis, we trained an autoencoder model. We then transferred its encoder weights following exactly the same protocol as before. We observe in this case that although the network learns faster in almost all the protocols in which weights are transferred, generalization does not improve, as in the classification-pretrained case. We conclude from this that faster learning can be attributed to the warm start provided by pretraining the network, but for better generalization, only the abstract scaffold results in transfer.

F. Parameters

We report here all the parameters required to reproduce the results. Table S1 recapitulates the parameters shared across all experiments in the manuscript, while Tables S2, S3 and S4 are specific to figures.

Table S1. Table of parameters shared across all experiments.

Description	Size
Network	
Learning rate	0.001
Batch size	1
Initialization	$\mathcal{U}\left(-\frac{1}{\sqrt{N}}, \frac{1}{\sqrt{N}}\right)$
Transfer function	ReLU
Loss function	Cross-Entropy
Data	
Number of unique letters per sequence m	2
Size of alphabet α	10

Table S2. Table of parameters for Fig. 1 of main text.

Description	Size
Network	
Number of input neurons	10
Number of recurrent neurons N (when not explicitly varied)	128
Number of output neurons	4
Number of training epochs	41
Data	
Sequence length L (when not explicitly varied)	6
Sequence cue length L_{cue}	4
Number of classes C (when not explicitly varied)	4
Fraction of train sequences	0.8
Max number of class combinations CC	20
Number of simulations with different initialization seeds	5

Table S3. Table of parameters for Figs. 2 and 3 of main text.

Description	Size
Network	
Number of input neurons	10
Number of recurrent neurons N	160
Number of output neurons	7
Number of training epochs	101
Data	
Sequence length L	4
Sequence cue length L_{cue}	4
Number of classes C	7
Fraction of train sequences	1.
Number of class combinations CC	1
Number of simulations with different initialization seeds	100

Table S4. Table of parameters Fig. 4 of main text and Figs. S1, S2, and S3.

Description	Size
Network	
Number of input neurons	10
Number of recurrent neurons N (when not explicitly varied)	160
Number of output neurons (class, pred)	5, 10
Number of training epochs (class, pred)	41, 101
Data	
Sequence length L (when not explicitly varied)	9
Sequence cue length L_{cue}	4
Number of classes C (when not explicitly varied)	5
Fraction of train sequences	0.8
Max number of class combinations CC	20
Number of simulations with different initialization seeds	5

Classification pretrained weights transferred to prediction task

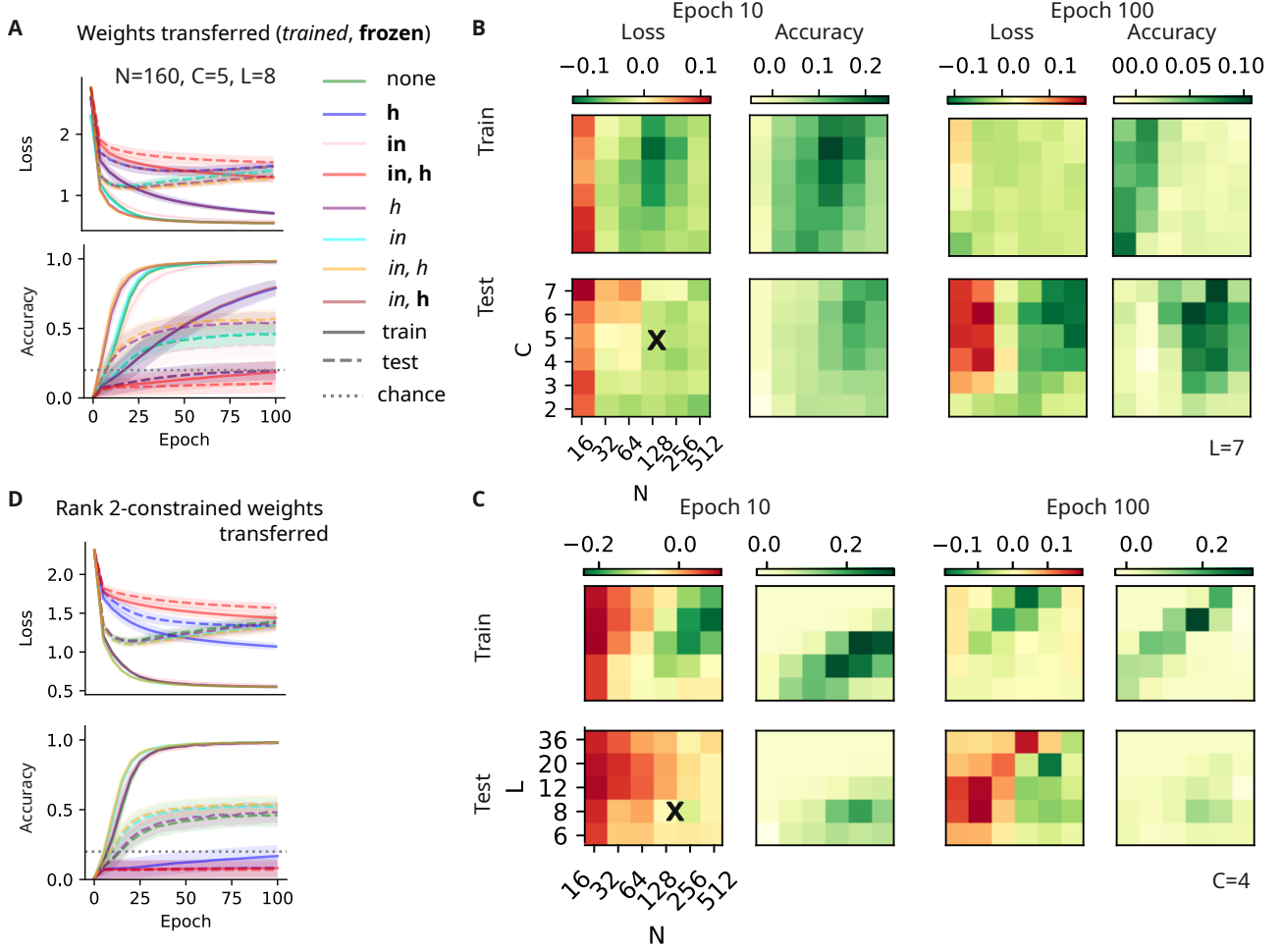


Figure S3. The effect of classification and reconstruction pretraining on prediction tasks. **A.** Loss (top) and accuracy (bottom) of prediction networks under three different initialization protocols: random initialization in the lazy regime (green), parameters from the classification pretraining transferred and kept frozen (blue, pink and red), and continued training (purple, cyan and orange). Finally, brown indicates frozen recurrent weights and retrained input weights. Lines indicate the mean across simulations; shaded areas show standard deviations. **B.** Difference in loss and accuracy between transfer-initialized and randomly initialized prediction networks, as a function of network size N (x-axis) and number of classes C (y-axis), for $L = 7$. These phase diagrams are plotted for early training (epoch 10) and late training (epoch 100). Green areas indicate improved performance through transfer. The black cross marks parameter configuration shown in panel A. **C.** Same as B but with sequence length L on the y-axis, for fixed number of classes $C = 4$. **D.** We pretrain rank-constrained networks on the classification task and then transfer the weights onto the prediction network following the same protocol as in A. **E.** We perform the same transfer experiments by pre-training on a reconstruction task, with the RNN as the encoder of an auto-encoder architecture. **F.** The loss and activity learning curves show that transfer makes it faster to learn the training data, but it does not lead to an improvement in the generalization performance.

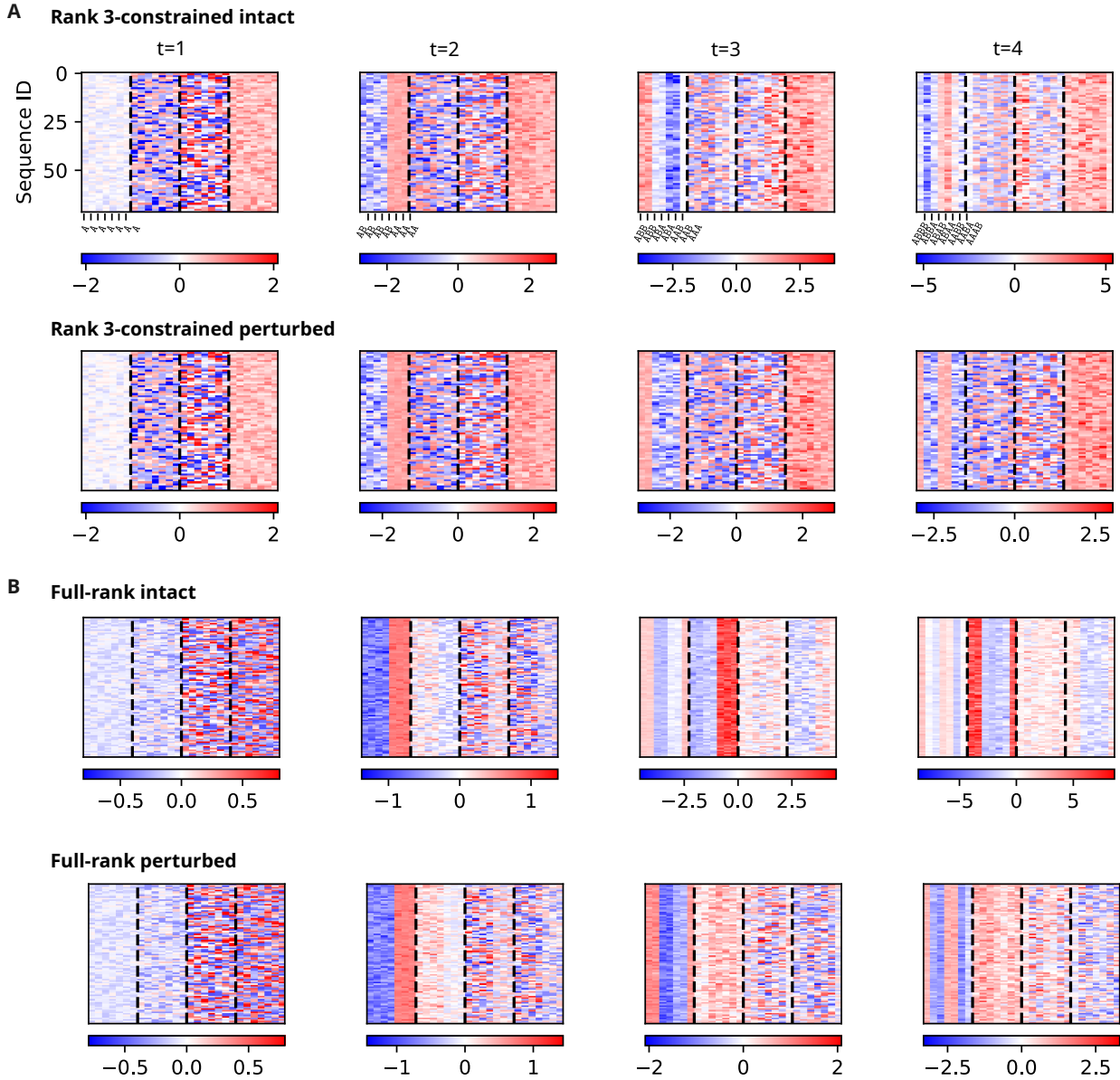


Figure S4. Perturbation experiments highlight the role of the singular components of the structured part of the recurrent connectivity. The RNN with a rank-3 recurrent weight matrix performs the task with high accuracy, approaching the performance of a full-rank one. In both cases, can see that the projections along the first principal components (A, top and B, top). By simulating a network where the dominant singular vector is removed from the recurrent weight matrix, we gain an insight into the role of the low-rank structured part (A, bottom and B, bottom). In both cases we notice the following. *i)* In the intact network the projections along the first singular components jointly carry information about the distinct classes into which the structure of sequences (or partial, processed so far) can be categorized. *ii)* When the dominant singular component is removed, the only projection that obviously carries information about the structure is SV1 itself: in particular, it is only informative of the latest seen transition (“same” vs “different”); since SV1 is removed from the recurrent weights, this is not propagated to the next time step, preventing the network from memorizing the whole structure.

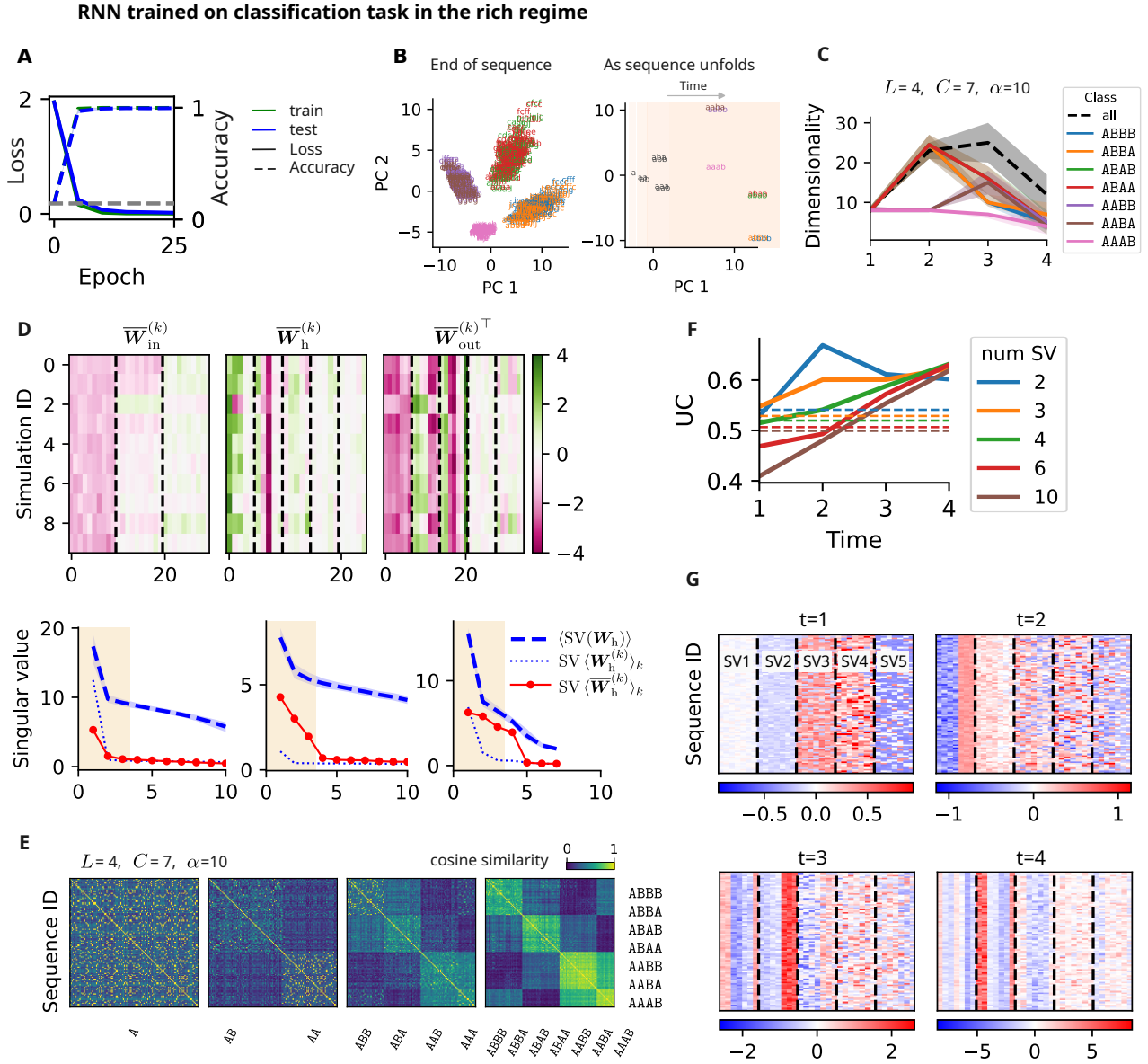


Figure S5. The results with “lazy” initialization regime hold in the “rich” case as well.